

Student Details

Name:	Mithielesh N
Roll Number:	DS23F3004299
Batch:	2023
Degree Level:	Diploma
Course:	Modern Application Development - 2

Project Details

Problem Statement

A full-stack, multi-role Applicant Tracking System (ATS) and campus placement portal (CareerOrbit) designed to streamline university recruitment. The application enables institute administrators to oversee global placement metrics and manage user directories, empowers recruiting companies to launch drives and manage a multi-stage candidate pipeline, and provides students with a smart job board to apply for roles, track their application status, and receive automated asynchronous updates.

Approach

Step 1: Analyzed the core requirements to establish a decoupled architecture. Opted to build a strict RESTful JSON API using Flask for the backend, paired with a modern Single Page Application (SPA) using the Vue 3 Composition API to handle complex dynamic states such as real-time ATS pipeline updates and smart job filtering without page reloads.

Step 2: Designed a highly normalized relational database schema using SQLAlchemy. The models include a unified User table (handling Role-Based Access Control for Admins, Companies, and Students), CompanyProfile, and PlacementDrive. The Application table acts as the associative entity securely mapping student profiles to specific company drives, tracking their exact stage in the hiring process. Targeted in-app alerts are managed via a dedicated JSON notification structure.

Step 3: Structured the Flask backend into modular controllers (auth.py, admin.py, company.py, user.py) to securely route traffic. Implemented robust backend validation such as strict numeric bounds for CGPA, automatic drive expiration checks, and duplication prevention to maintain absolute database integrity and secure session management.

Step 4: Developed the frontend interface. Utilized Vue.js reactive variables to implement instant search filtering for the Admin directory, an auto-categorizing student job board (sorting drives into Active, Upcoming, Missed, and Applied), and dynamic modal-based operations for interview scheduling and pipeline progression

Step 5: Integrated background job processing to handle heavy lifting without blocking the UI. Configured Celery and Redis to execute user-triggered CSV data exports asynchronously.

Step 6: Implemented scheduled tasks using Celery Beat. Engineered cron-like jobs to automatically scan the database to send daily in-app reminders to students regarding upcoming application deadlines, as well as compile and distribute comprehensive HTML campus placement activity reports on the first day of every month.

Step 7: Optimized backend performance by integrating Flask-Caching with Redis. High-traffic endpoints (such as fetching the master list of approved job drives and calculating global admin dashboard metrics) are cached in-memory to drastically reduce SQLite database queries.

Step 8: Engineered a comprehensive Analytics Dashboard. Integrated Chart.js to consume JSON API data and render real-time visual insights. This includes campus-wide salary distribution bar charts for the Admin dashboard and interactive, multi-stage recruitment funnel doughnut charts for Company recruiters to track application conversion rates.

Frameworks and Libraries used

1. **SQLite:** The lightweight, file-based relational database engine used for persisting all user profiles, academic hierarchies, and historical score data.
2. **Flask:** The core Python micro-framework utilized to construct the RESTful backend API and serve the initial SPA template.
3. **Vue.js (via CDN):** The progressive JavaScript framework used to build the reactive Single Page Application (SPA), managing complex client-side state and DOM updates seamlessly.
4. **Celery:** A distributed asynchronous task queue used to offload long-running processes (like CSV exports) and execute periodic scheduled jobs (monthly reports).
5. **Redis:** Serves a dual architectural purpose as the high-speed, in-memory message broker for Celery workers and the backend storage engine for Flask API caching.
6. **Bootstrap 5:** The responsive CSS framework utilized to design a modern, mobile-friendly UI with pre-built components like modals, grids, and badges.
7. **Flask-SQLAlchemy:** An extension that bridges Python objects to SQLite tables via an Object-Relational Mapper (ORM), ensuring secure, sanitized, and efficient database queries.
8. **Flask-Login:** Provides secure user session management, handling authentication state and enforcing Role-Based Access Control (RBAC) across protected API routes.
9. **Flask-Caching:** An optimization extension that dramatically reduces database load by serving frequently requested, relatively static data directly from Redis memory.

10. **Chart.js:** A client-side JavaScript charting library integrated to render dynamic, visually rich analytics dashboards directly in the browser.
11. **Werkzeug Security:** Utilized under the hood for the cryptographic hashing (Bcrypt) and verification of user passwords to ensure strict data security.

Presentation Video (Drive Link)

ER Diagram

